

Задание 2. Создание приложения с распараллеливанием задач

Аннотация: В данном практическом занятии будет рассмотрен процесс создания простого приложения с использованием многозадачности.

1. Создадим многозадачное консольное приложение и назовем его, к примеру, "SimpleMultitaskingApplication":

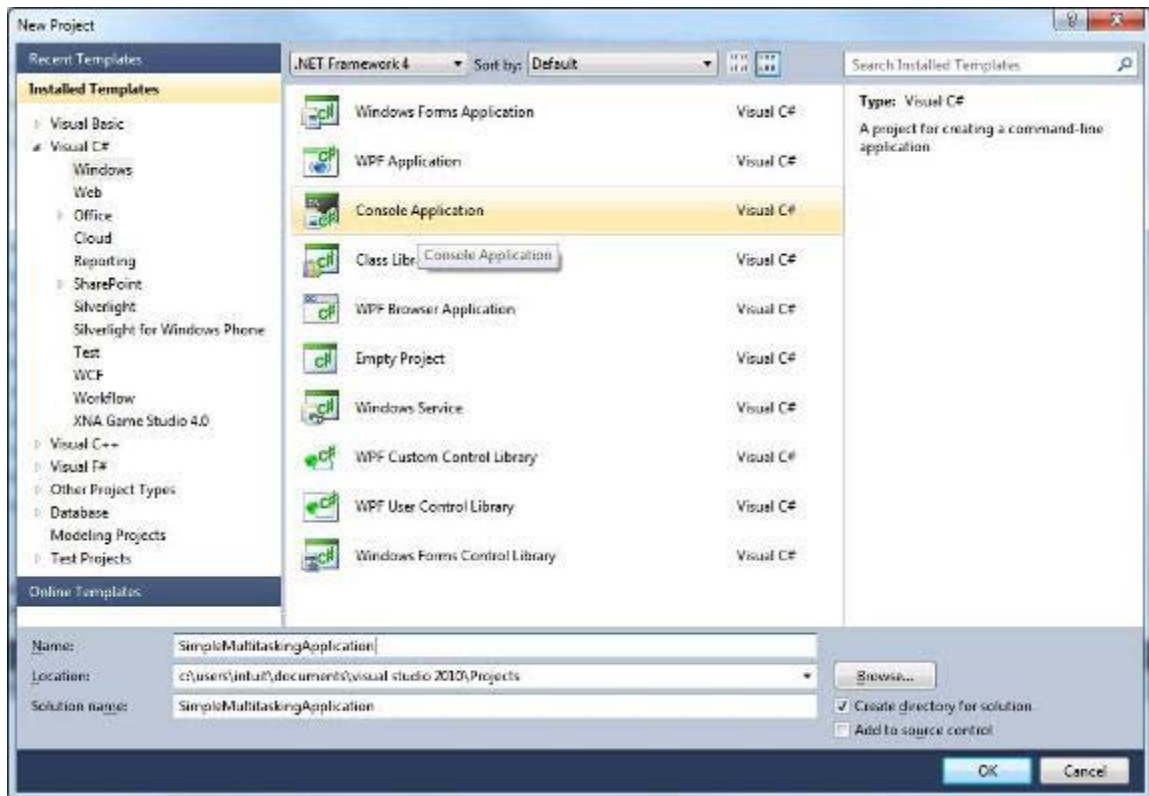


Рис. 1.

2. Создадим статичный метод, который будет выводить на консоль сообщение "Hello world!":

```
private static void printMessage() {  
    Console.WriteLine("Hello world!");  
}
```
3. Создадим четыре различных задачи, различными способами:

```
Task task1 = new Task(new Action(printMessage));  
Task task2 = new Task(delegate { Console.WriteLine("Hello  
world!"); });  
Task task3 = new Task(() => printMessage());  
Task task4 = new Task(() => { Console.WriteLine("Hello  
world!"); });
```

 - o task1 - используем делегат Action и именной метод (в нашем случае printMessage);
 - o task2 - используем анонимный делегат;
 - o task3 - используем лямбда выражение и именной метод;
 - o task4 - используем лямбда выражение и анонимный метод.
4. Запускаем задачи с помощью метода Start():

```
task1.Start()  
task2.Start()  
task3.Start()  
task4.Start()
```

5. Обозначим завершение главного потока (метод `Main()`) с помощью фрагмента кода:

```
Console.WriteLine("Главный метод завершен.")
    Console.ReadLine()
```

Листинг кода программы:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace SimpleMultitaskingApplication
{
    class Program
    {
        private static void printMessage()
        {
            Console.WriteLine("Hello world!");
        }

        static void Main(string[] args)
        {
            Task task1 = new Task(new Action(printMessage));
            Task task2 = new Task(delegate { Console.WriteLine("Hello
world!"); });
            Task task3 = new Task(() => printMessage());
            Task task4 = new Task(() => { Console.WriteLine("Hello
world!"); });
            task1.Start();
            task2.Start();
            task3.Start();
            task4.Start();
            Console.WriteLine("Главный метод завершен.");
            Console.ReadLine();
        }
    }
}
```

6. Запустим программу, результат выглядит следующим образом:

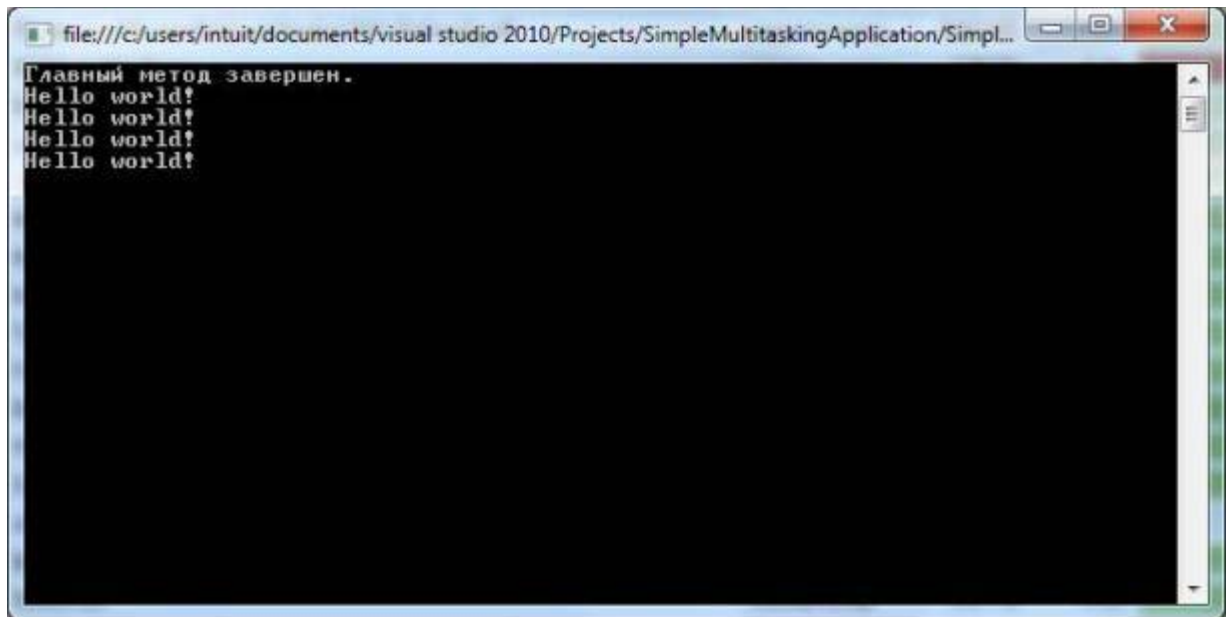


Рис. 2.

7. Модифицируем программу. Изменим метод `printMessage()`:

```
private static string printMessage(string message)
{
    return message.ToUpper();
}
```

Данный перегруженный метод теперь возвращает значение типа `string`, преобразованное в верхний регистр.

8. Создадим задачу, в методе `Main`, которая бы передавала в метод `printMessage` значение и возвращала результат выполнения задачи в виде строки:

```
Task <string> message = new Task<string>(mes => printMessage((string)
mes), "hello world")
message.Start()
Console.WriteLine("Сообщение: " + message.Result)
```

9. Запустим программу. Программа выведет следующий результат на экран:

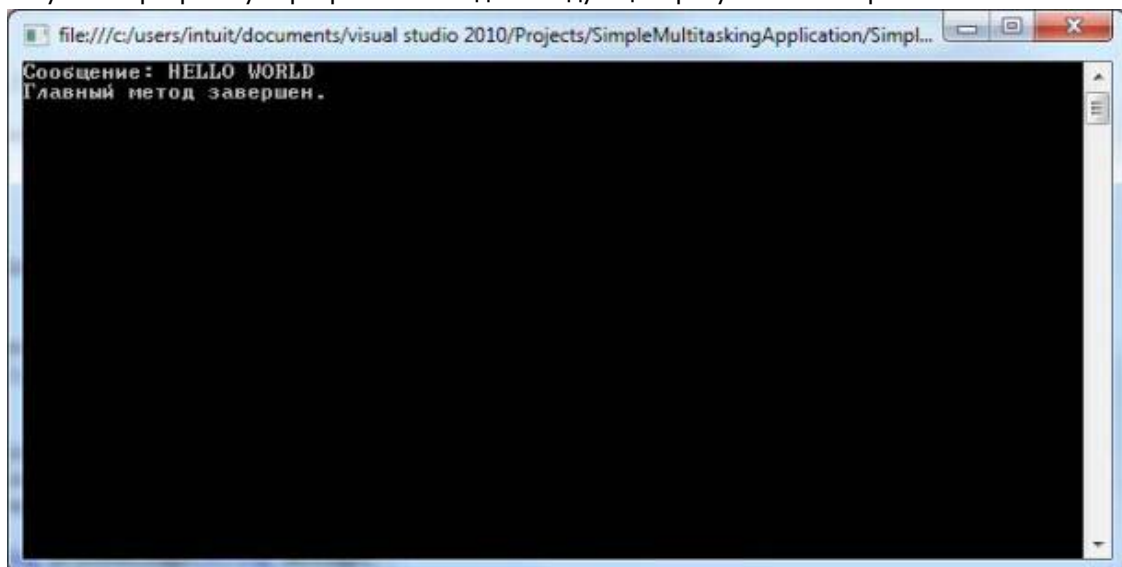


Рис. 3.

Листинг кода программы:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace SimpleMultitaskingApplication
{
    class Program
    {
        static void Main(string[] args)
        {
            Task<string> message = new Task<string>(mes =>
printMessage((string)mes), "hello world");
            message.Start();
            Console.WriteLine("Сообщение: " + message.Result);
            Console.WriteLine("Главный метод завершен.");
            Console.ReadLine();
        }
        private static string printMessage(string message)
        {
            return message.ToUpper();
        }
    }
}
```

10. Теперь, создадим новую задачу, с использованием метода `ContinueWith`. Данный метод вызывает продолжение выполнения целевой задачи, но уже в другой задаче, в нашем случае задачи `message`:

```
Task cwt =message.ContinueWith(task => Console.WriteLine("Сообщение: " + message.Result));
```

11. Запустим приложение. Результат будет следующим:

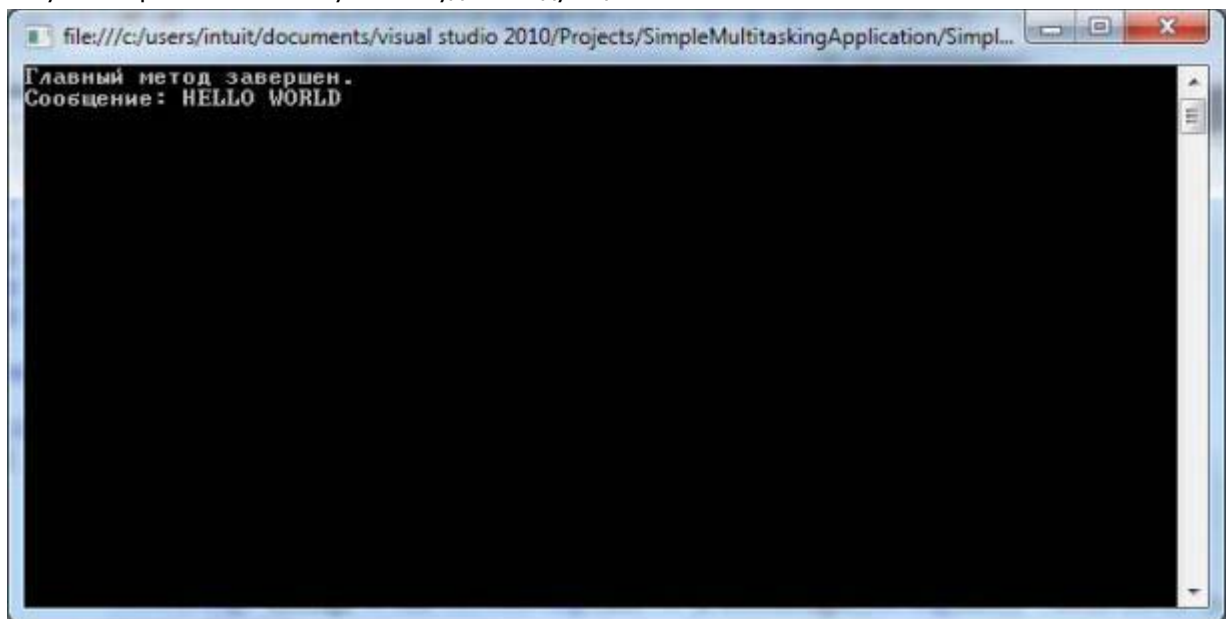


Рис. 4.

Примечание. Как видно из результата выполнения программы, главный метод завершит выполнение раньше, чем дочерни задачи, это связано с тем, что задача вывода на экран

результата (Task cwt) выполнения метода printMessage., выполнится только тогда, когда завершится выполнение задачи Task <string> message.

Листинг кода программы:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace SimpleMultitaskingApplication
{
    class Program
    {
        private static string printMessage(string message)
        {
            return message.ToUpper();
        }

        static void Main(string[] args)
        {
            Task<string> message = new Task<string>(mes =>
printMessage((string)mes), "hello world");
            message.Start();
            Task cwt = message.ContinueWith(task =>
Console.WriteLine("Сообщение: " + message.Result));
            Console.WriteLine("Главный метод завершен.");
            Console.ReadLine();
        }
    }
}
```

12. Для создания дочерних задач используем следующий фрагмент кода:

```
Task<string[]> message = new Task<string[]>(() =>
{
    var result = new string[3];
    new Task(() => result[0] = printMessage("Hello"),
TaskCreationOptions.AttachedToParent).Start();
    new Task(() => result[1] = printMessage("World"),
TaskCreationOptions.AttachedToParent).Start();
    new Task(() => result[2] = printMessage("Hello world!"),
TaskCreationOptions.AttachedToParent).Start();
    return result;
});
```

Данный фрагмент кода возвращает строковый массив, который формируется с помощью дочерних задач.

13. Теперь необходимо вывести строковый массив на экран с помощью фрагмента кода:

```
var cwt =message.ContinueWith(mes =>
Array.ForEach(mes.Result, Console.WriteLine))
message.Start()
```

14. Запустим программу. В результате на экране отобразится следующее:

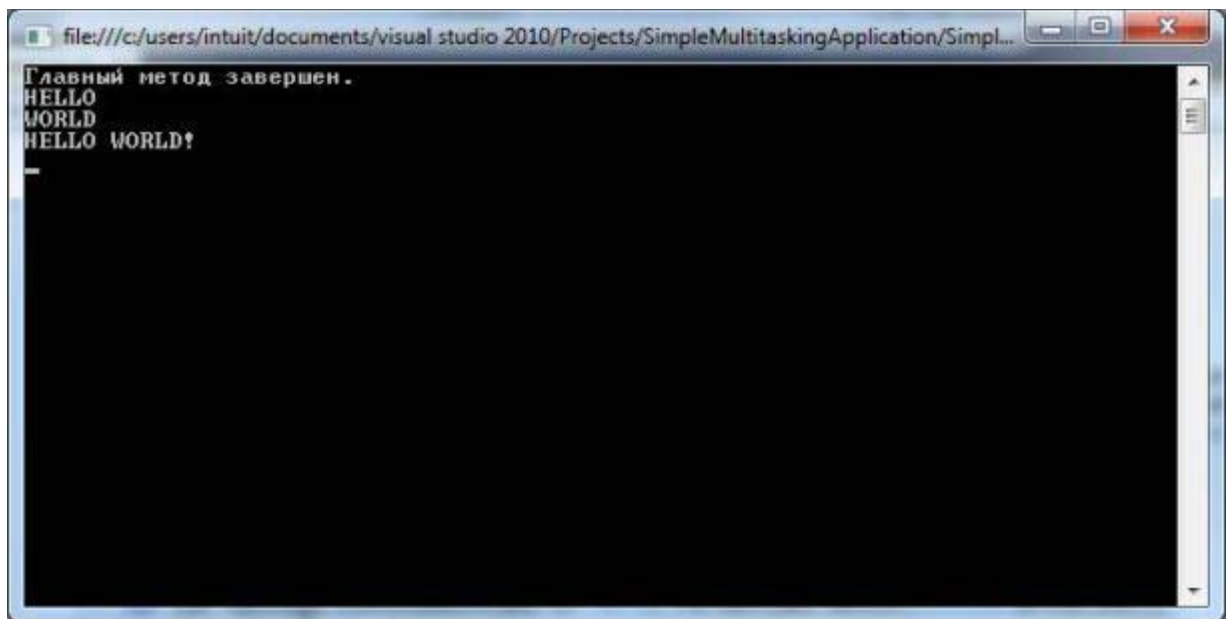


Рис. 5.

Листинг кода программы:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace SimpleMultitaskingApplication
{
    class Program
    {
        static void Main(string[] args)
        {
            Task<string[]> message = new Task<string[]>(() =>
            {
                var result = new string[3];
                new Task(() => result[0] = printMessage("Hello"),
TaskCreationOptions.AttachedToParent).Start();
                new Task(() => result[1] = printMessage("World"),
TaskCreationOptions.AttachedToParent).Start();
                new Task(() => result[2] = printMessage("Hello
world!"), TaskCreationOptions.AttachedToParent).Start();
                return result;
            });
            var cwt =message.ContinueWith(mes =>
Array.ForEach(mes.Result, Console.WriteLine));
            message.Start();
            Console.WriteLine("Главный метод завершен.");
            Console.ReadLine();
        }
        private static string printMessage(string message)
        {
            return message.ToUpper();
        }
    }
}
```